



It's Your Life

with









그란데 말입니다



Props

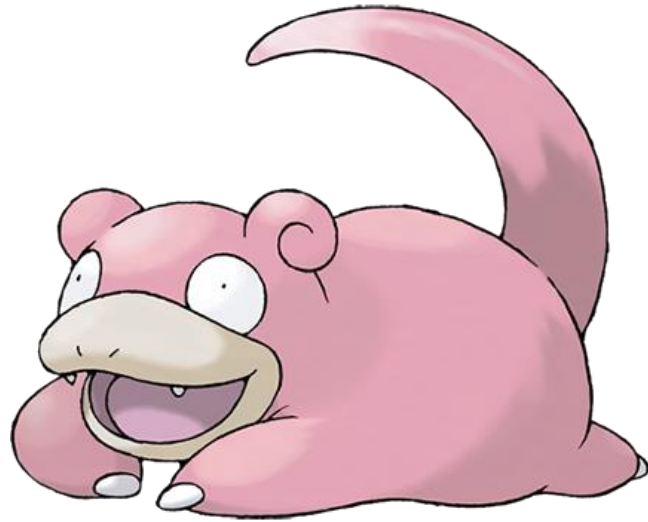




혹시, html 요소는 못 보내나요!?



- 이렇게 데이터 말고 html 요소나 아예 다른 컴포넌트를 자식 컴포넌트에 전달할 수는 없나요!?
- 그럼 좀 더 변화무쌍한 컴포넌트를 만들 수 있을 것 같은데요!?







Vue

Slot

모달 창을 아시나요?!



Please write a review for better service and other users.

booking applicant information

booking number 367559587

5-6시



휴지통을 비우시겠습니까?

휴지통의 모든 메일은 영구 삭제되어
복구할 수 없습니다.

취소

확인

ver Booking Home

For any inquiries regarding the services and other matters, please contact the
customer center(+82.1644.5690).

Terms and conditions | Privacy policy | NAVER Booking customer service center

Copyright © NAVER Corp. All Rights Reserved.

1

2

3

4

5

Title

Contents

Button

Button

모달 창의 내용은 물론
이런 버튼이 1개 일 때!? 2개 일 때?
혹은 그림을 넣고 싶을 때!?

Props 만으로 구현이 쉬울까요?



자식 컴포넌트한테 HTML 을 보내고 싶다면?



Slot(머신 아닙니다)에 무엇을 넣느냐에 따라!

- 모달에 어떤 HTML 을 전달하느냐에 따라서 모달의 내용은 완전히 달라지게 될 것입니다!
- 물론 props 만으로도 구현이 가능하지만, props 를 받아서 HTML 을 다시 그려야 하므로 귀찮습니다
- 그러니 바로 HTML 을 전달 하시면!? → 편합니다

부모 컴포넌트

```
<template>
  <div>
    <SlotChild>
      <!-- 자식 요소의 HTML 후손 요소로 보내고 싶은 것들을 보냅니다! -->
      <h1>테스트</h1>
    </SlotChild>
  </div>
</template>
```



자식 컴포넌트

```
<template>
  <div>
    <slot><!-- 여기에 부모가 보낸 요소가 들어 갑니다! --></slot>
  </div>
</template>
```

Slot 만들기, 자식 컴포넌트에 만들어 줍니다!



```
<template>
  <div>
    <slot><!-- 여기에 부모가 보낸 요소가 들어 갑니다! --></slot>
  </div>
</template>

<script>
export default {
  name: 'SlotChild',
};
</script>
```

부모로부터 전달 받은 HTML 요소를
넣어줄 <slot></slot> 공간
만들어 주기!

부모에서 slot 에 HTML 전달하기!



```
<template lang="">
  <div>
    <SlotChild>
      <!-- 자식 요소의 HTML 후손 요소로 보내고 싶은 것들을 보냅니다! -->
      <h1>테스트</h1>
      
    </SlotChild>
  </div>
</template>

<script>
import SlotChild from './SlotChild.vue';

export default {
  name: 'SlotParent',
  components: { SlotChild },
};
</script>
<style lang=""></style>
```

<SlotChild /> 의
<slot>에 넣고 싶은
HTML 요소를
<SlotChild /> 의
후손 요소로 전달



테스트



테스트2

인풋도 들어 갈 수 있다구욏!

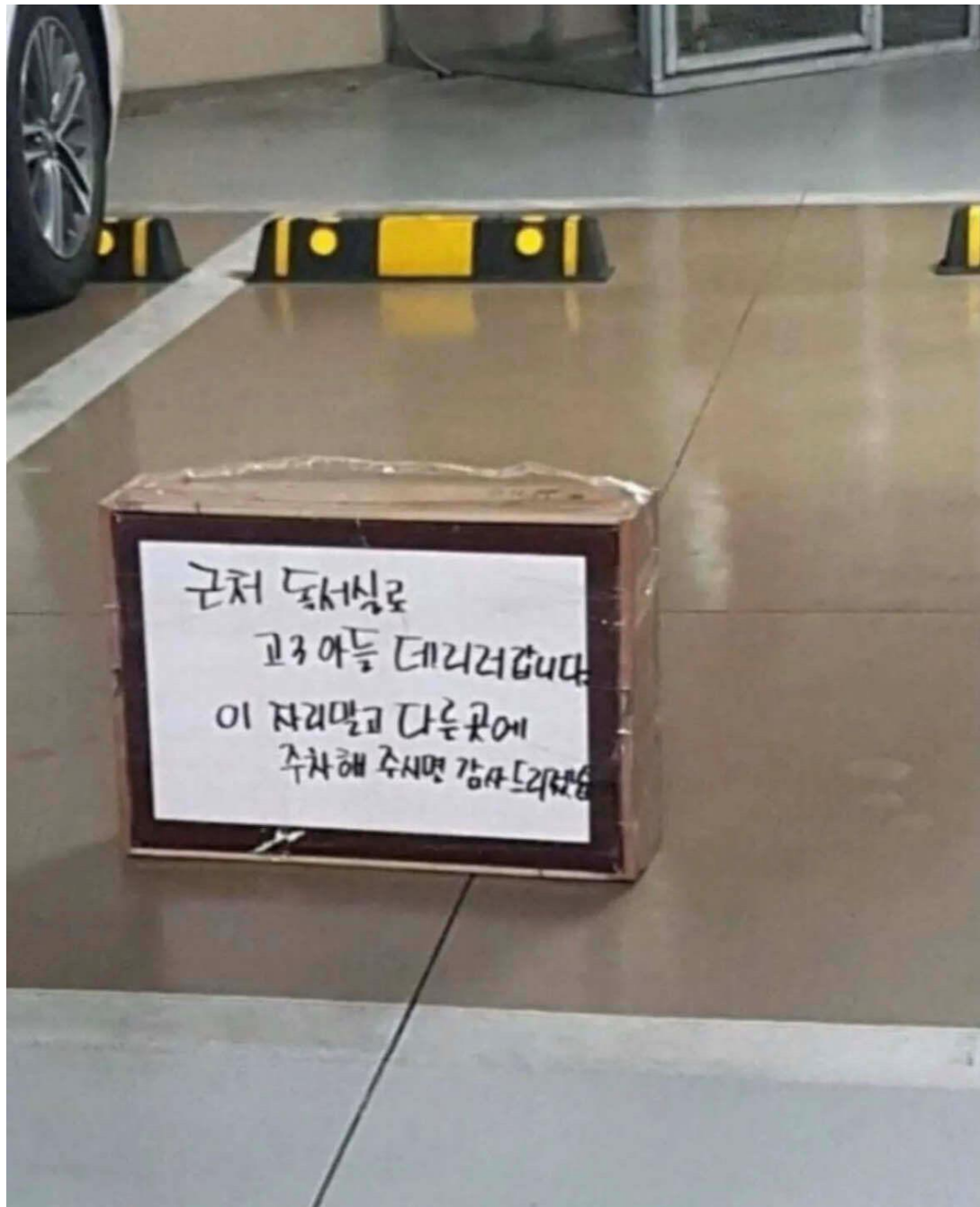
달기

달지 마요 ㅠㅠ



Named

Slot



Slot 에 이름을 붙여서 원하는 슬롯을 선택!



- slot 을 여러 개 사용하는 경우 어떤 부모가 보낸 요소를 어떤 slot 에 보내야 할까요!?
- 그럴 때는 slot 에 이름을 붙여서 원하는 slot 을 선택할 수 있습니다!



자식 컴포넌트



```
<template>
  <div>
    <div class="top">
      <h1>뭐든지 기울이는 TOP 슬롯 입니다!</h1>
      <slot name="top"></slot>
    </div>
    <div class="bottom">
      <h1>뭐든지 밑줄치는 BOTTOM 슬롯 입니다!</h1>
      <slot name="bottom"></slot>
    </div>
  </div>
</template>

<style scoped>
.top {
  font-style: italic;
  background-color: skyblue;
}
.bottom {
  text-decoration: underline;
  background-color: orange;
}
</style>
```

slot 인데 '이름'이
붙어 있습니다!

slot 를 감싸고 있는
div 요소는 각각의 스타일이
적용되어 있습니다



부모 컴포넌트

```
<template>
  <div>
    <NamedSlotChild>
      <template v-slot:top>
        <h2>여기는 top 으로 보내는 html 요소 입니다!</h2>
      </template>
      <template #bottom>
        <h2>요거는 bottom 으로 보내는 html 요소 입니다!</h2>
      </template>
    </NamedSlotChild>
  </div>
</template>

<script>
import NamedSlotChild from './NamedSlotChild.vue';

export default {
  name: 'NamedSlotParent',
  components: { NamedSlotChild },
};
</script>
```

원하는 slot 은
v-slot:슬롯명
디렉티브를 사용하여
연결 합니다

v-slot 은 # 으로
축약 가능!!

뭐든지 기울이는 **TOP** 슬롯 입니다!

여기는 *top* 으로 보내는 *html* 요소 입니다!

뭐든지 밑줄치는 **BOTTOM** 슬롯 입니다!

요거는 *bottom* 으로 보내는 *html* 요소 입니다!



```
<slot name='top' />
```



```
<slot name='bottom' />
```



헤더 영역

컨텐츠 영역

컨텐츠 영역

컨텐츠 영역

컨텐츠 영역

컨텐츠 영역

컨텐츠 영역

사이드

사이드

사이드

Footer text

실습, Slot 활용!



- Vue Slot 을 활용하여 <FancyPhotoBox /> 만들기
- <FancyPhotoBox /> 는 사진의 제목을 넣는 title slot 과 사진을 넣는 photo slot 이 존재 합니다!
- <FancyPhotoBox /> 를 호출한 부모는 <FancyPhotoBoxParent /> 로 만들어 주시고 <FancyPhotoBox /> 에게 <h1> 태그로 사진의 제목을 보내주고, 태그로 사진을 직접 전달해 줍니다!

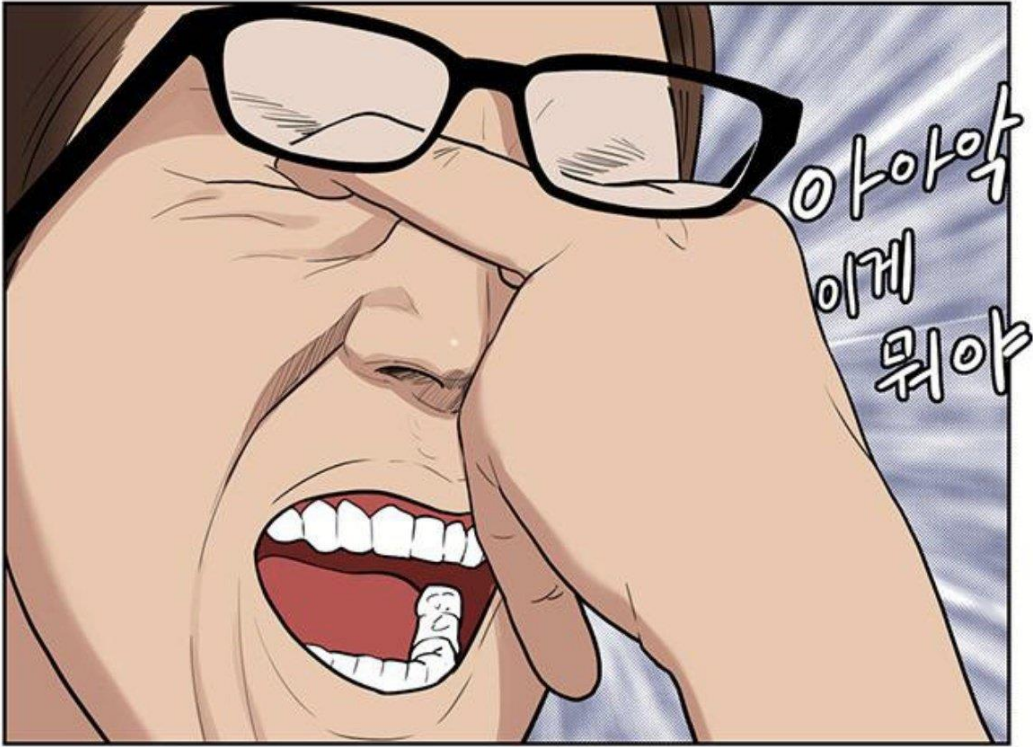
실습, Slot 활용!



- 각자의 디자인적 감각을 사용해서 사진의 제목과 이미지를 넣어주는
<FancyPhotoBox /> 를 완성하세요!

```
<template>
  <div>
    <FancyPhotoBox>
      <template v-slot:title>
        <h1>지겨운 장동민 마멋</h1>
      </template>
      <template v-slot:photo>
        
      </template>
    </FancyPhotoBox>
  </div>
</template>
```

FancyPhotoBoxParent



항마력 테스트

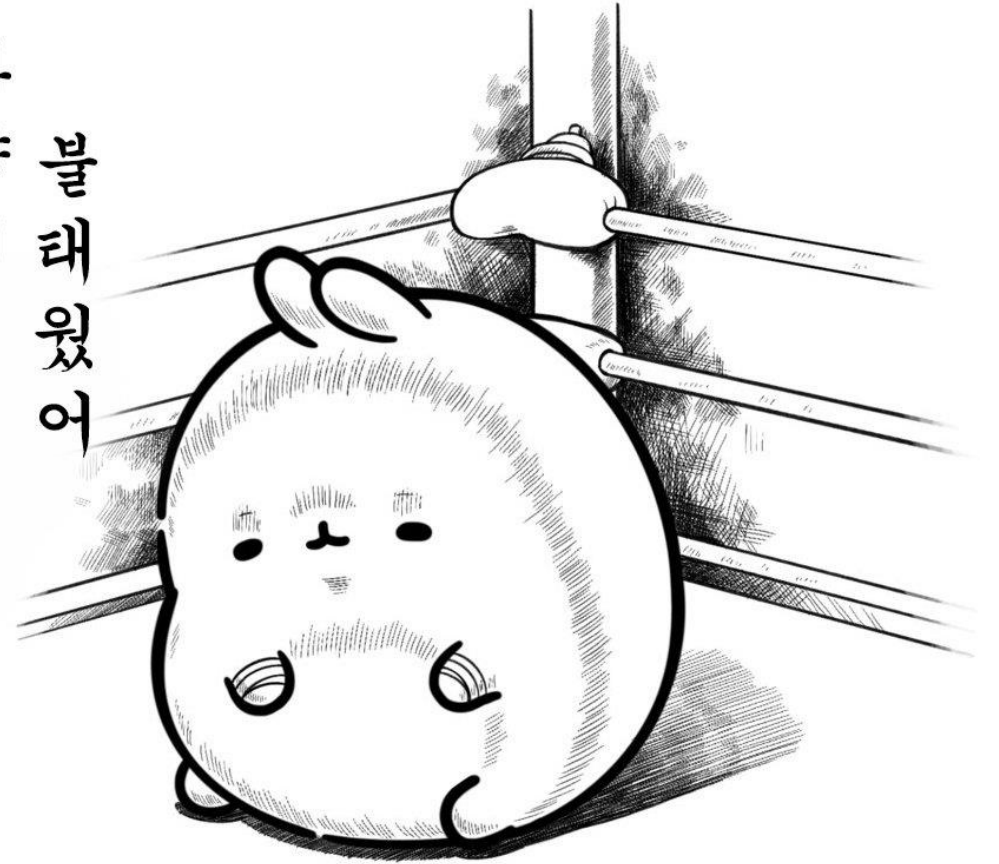




지겨운 장동민 마멋



하
양
게
불
태
웠
어





Scoped Slot



지미🌹

@LOV3_OR_NOT

강사님...
저는 자꾸 왜 받기만 해야하나요?

제가 보내면 안되나요!?



자식 컴포넌트



안 돼 안 바꿔줘. 바꿀 생각 없어. 빨리 돌아가



사실 생각해 보면!?



- 자식이 부모한테 HTML 을 보낼 필요가 없습니다! 강 자기한테 그리면 되니까요! 😊
- 대신, data 같은 경우는 이야기가 달라지는데요.
- 데이터를 자식이 관리하고 해당 데이터를 어떻게 그려줄 것인지를 부모가 결정하는 케이스가 있을 수 있습니다! 이런 경우에서 사용하는 기술을 한 번 확인해 보시죠!



자식 컴포넌트

```
<template>
  <div>
    <slot v-bind:msg="data1"></slot>
    <slot name="named" v-bind:msg="data2"></slot>
  </div>
</template>
```

```
<script>
export default {
  name: 'ScopedSlotChild',
  data() {
    return {
      data1:
        '과연 나는 어떤 태그에 들어갈까? 와아아아아 너어어어어무 궁금하다 😊',
      data2: '그럼 나는!? 😊',
    };
  },
};
</script>
```

자식 컴포넌트에 존재하는
data1, data2 를
속성 이름을 임의로 정하여
부모 요소로 전달!



부모 컴포넌트

```
<template>
  <div>
    <ScopedSlotChild v-slot:default="fromChild">
      <h1>{{ fromChild.msg }}</h1>
    </ScopedSlotChild>
    <ScopedSlotChild v-slot:named="fromChild2">
      <h1>{{ fromChild2.msg }}</h1>
    </ScopedSlotChild>
  </div>
</template>

<script>
import ScopedSlotChild from './ScopedSlotChild.vue';
export default {
  name: 'ScopedSlotParent',
  components: { ScopedSlotChild },
};
</script>
```

v-slot 디렉티브를 이용하여
이름 없는(=디폴트) 슬롯에서
올라온 데이터를

임의의 변수 이름을 지어주고
객체 데이터로 받기

자식 요소에서 전달한
msg 라는 이름으로 전달한
데이터를 꺼내서 사용

& 내가 원하는 태그에 담아서
다시 자식으로 전달!
즉, 자신이 원하는 태그로
자유롭게 전달 가능!



```
<ChildScopedSlot v-slot:default="fromChild">  
  <h1>{{ fromChild.msg }}</h1>  
</ChildScopedSlot>
```

과연 나는 어떤 태그에 들어갈까? 와아아아아 너어어어어무 궁금하다 🤪

```
<ScopedSlotChild v-slot:named="fromChild2">  
  <p>{{ fromChild2.msg }}</p>  
</ScopedSlotChild>
```

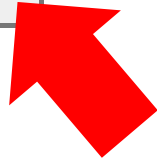
그럼 나는!? 😊



Vue Slot 을 이용한 모달 만들기



모달 열기



Modal

Lorem ipsum dolor sit, a

모달 열기

이 모달을 당장 닫지 않으면!?

씨끄러울 겁니다!





```
<template>
  <div class="modal-overlay" v-if="visible">
    <div class="modal">
      <button class="close-button" @click="sendClose">X</button>
      <div class="modal-content">
        <slot></slot>
      </div>
    </div>
  </div>
</template>
<script>
export default {
  name: 'Modal',
  props: {
    visible: {
      type: Boolean,
      required: true,
    },
  },
  methods: {
    sendClose() {
      this.$emit('close');
    },
  },
};</script>
```

부모로부터 받은 props 값으로
해당 모달을 보여줄지 여부를
결정 합니다!

자식 컴포넌트(Modal.vue)



```
<template>
  <div>
    <button @click="showModal">모달 열기</button>
    <Modal :visible="isModalVisible" @close="hideModal">
      <h2>이 모달을 당장 닫지 않으면!?!</h2>
      <p>씨끄러울 겁니다!</p>
      
    </Modal>
  </div>
</template>
```

부모에서 관리하는
isModalVisible 데이터를
props 로 전달하여
모달의 가시성을 관리

```
<script>
import Modal from './Modal.vue';

export default {
  name: 'ModalContainer',
  components: {
    Modal,
  },
  data() {
    return {
      isModalVisible: false,
    };
  },
}
```

부모 컴포넌트(ModalParent.vue)



```
<template>
  <div class="modal-overlay" v-if="visible">
    <div class="modal">
      <button class="close-button" @click="sendClose">X</button>
      <div class="modal-content">
        <slot></slot>
      </div>
    </div>
  </div>
</template>
<script>
export default {
  name: 'Modal',
  props: {
    visible: {
      type: Boolean,
      required: true,
    },
  },
  methods: {
    sendClose() {
      this.$emit('close');
    },
  },
};</script>
```

x 버튼을 누르면
sendClose 메서드가 실행되고
sendClose 는 부모 컴포넌트에게
close 이벤트를 emit 합니다!

자식 컴포넌트(Modal.vue)



```
<template>
  <div>
    <button @click="showModal">모달 열기</button>
    <Modal :visible="isModalVisible" @close="hideModal">
      <h2>이 모달을 당장 닫지 않으면!?!</h2>
      <p>씨끄러울 겁니다!</p>
      
    </Modal>
  </div>
</template>
```

자식 컴포넌트에서
close 라는 Event 가 emit 되면
hideModal 메서드를 실행

```
<script>
import Modal from './Modal.vue';

export default {
  methods: {
    showModal() {
      this.isModalVisible = true;
    },
    hideModal() {
      this.isModalVisible = false;
    },
  },
}
```

부모 컴포넌트(ModalParent.vue)

```
<template>
  <div>
    <button @click="showModal">모달 열기</button>
    <Modal :visible="isModalVisible" @close="hideModal">
      <h2>이 모달을 당장 닫지 않으면!?!</h2>
      <p>씨끄러울 겁니다!</p>
      
    </Modal>
  </div>
</template>
```

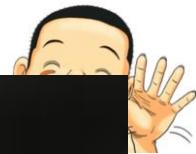
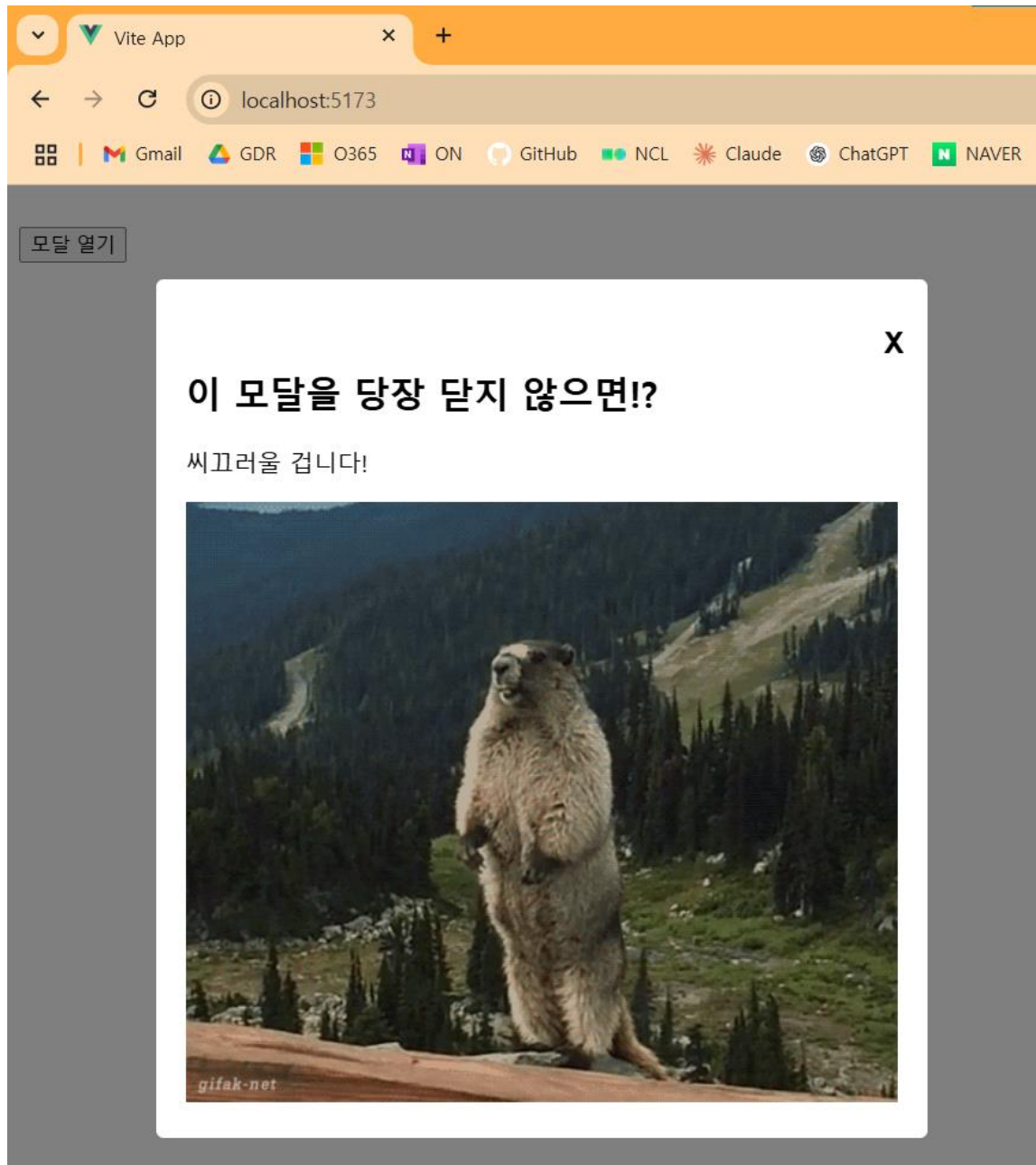
```
<script>
import Modal from './Modal.vue';

export default {
  methods: {
    showModal() {
      this.isModalVisible = true;
    },
    hideModal() {
      this.isModalVisible = false;
    },
  },
}
```

버튼을 클릭하면
showModal 이 실행되고
isModalVisible 데이터의 값을 변경

변경된 isModalVisible 값은
다시 props 로 전달 되고
변경 된 값을 전달 받은
Modal 의 v-if 가 작동하여
모달이 뜨는 구조!

부모 컴포넌트(ModalParent.vue)



Teleport!



특정 요소를
내가 원하는 곳으로 보내서
화면에 띄고 싶을 때 쓰는
기술입니다!



두산 13
LG 5

▶ **장사후 Naver 채널 고정!**



```
<template>
  <button @click="teleportTo('orange')">오렌지로</button>
  <button @click="teleportTo('skyblue')">하늘색으로</button>
  <button @click="teleportTo('red')">빨강으로</button>
  <div id="orange"></div>
  <div id="skyblue"></div>
  <div id="red"></div>

  <teleport :to="teleportTarget">
    <h1>나는 어디로 가는가? 뭘 하고 살아야 하나</h1>
  </teleport>
</template>
```

화면 어디로든 보내고 싶은 요소를
<teleport> 의 후손으로 작성!

보낼 곳은 to 의 값으로 전달하면
됩니다! (= CSS 의 선택자)



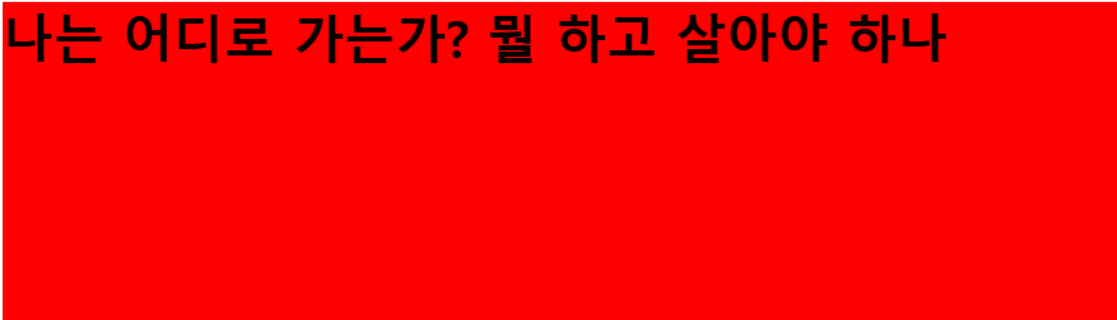
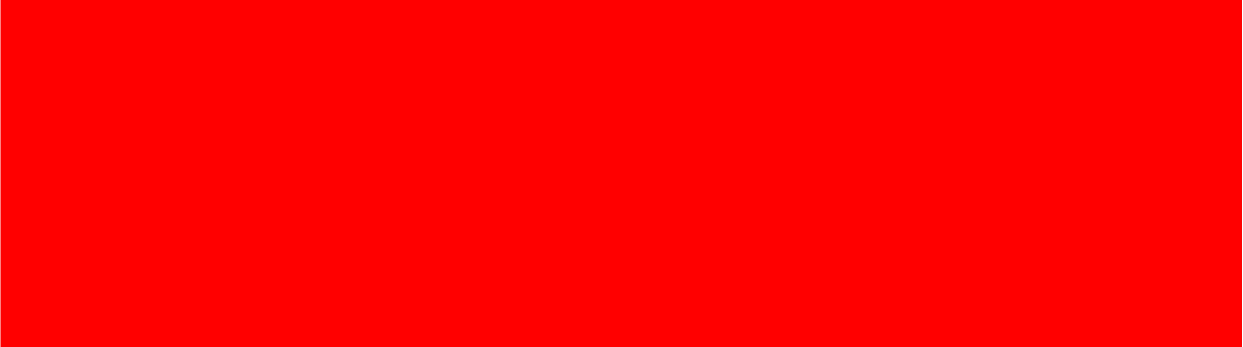
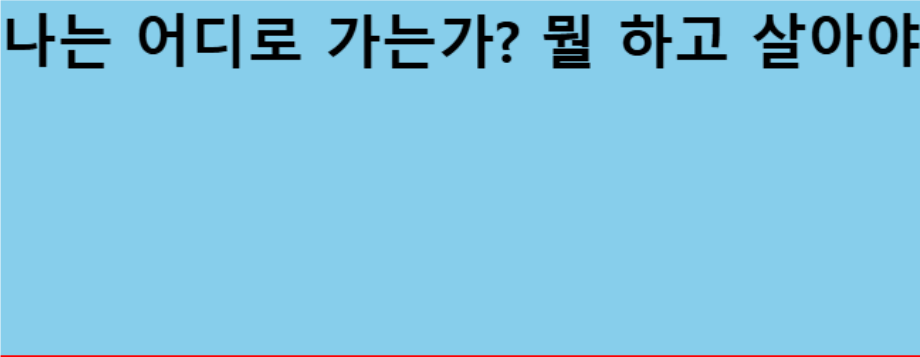
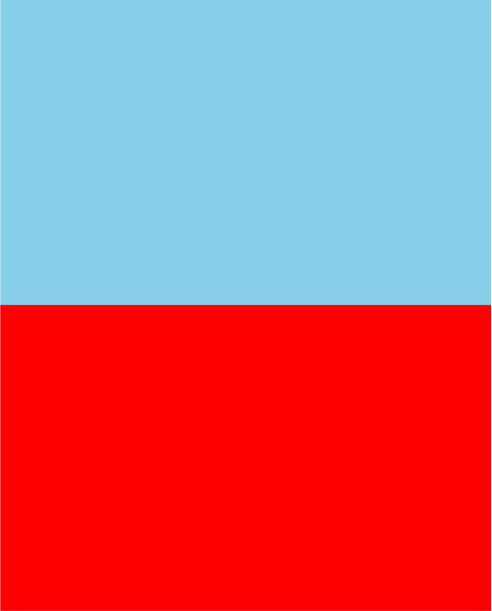
메서드를 사용하여
각각의 색상이 있는 공간으로
<h1> 태그를 옮겨볼 겁니다!



```
<script>
export default {
  name: 'Teleport',
  data() {
    return {
      teleportTarget: 'body',
    };
  },
  methods: {
    teleportTo(zone) {
      this.teleportTarget = `#${zone}`;
    },
  },
};
</script>
```

teleportTo 메서드가 실행 되면
각각의 버튼에서 전달한 색상 값으로
teleportTarget 이 변경 되고
해당 target 으로
H1 태그가 이동 합니다!

나는 어디로 가는가? 뭘 하고 살아야 하나



나는 어디로 가는가? 뭘 하고 살아야 하나



금강주택

시간을 따지는 아파트 -
금강펜테리움

천호엔케이

천호

정답 261

두산 13 LG 5

▶ **장시후 LIVE! 채널 고정!**

